Cross-Queue `XDP_REDIRECT` to AF_XDP Sockets with Shared UMEM in Zero-Copy Mode

Analysis of the `xsk_rcv_check()` Guard and Proposal for Relaxation

Claude Code (Anthropic)

by request and with guidance of Alex Dvoretsky

April 2026

Abstract

AF_XDP sockets bound to different hardware queues on the same NIC cannot receive cross-queue redirected frames, even when sharing a single UMEM region. The kernel enforces a `queue_id` equality check in `xsk_rcv_check()` (net/xdp/xsk.c) that silently drops any frame whose receive queue index differs from the target socket's bound queue. This paper traces the complete redirect code path—from driver NAPI handler through the generic XSK layer—and demonstrates that for aligned shared-UMEM configurations, the check is the *sole* barrier to correct cross-queue delivery. No driver modifications are required. We address concerns regarding ring index management, buffer ownership transfer, DMA coherency, teardown safety, and pool accounting, showing that each is handled correctly by existing code. We propose a minimal kernel patch to relax the guard for sockets sharing the same UMEM, enabling hardware-queue-level packet steering within a single AF_XDP application.

Contents

1 Motivation **2**
- 1.1 The Multi-Queue AF_XDP Router . . . 2
- 1.2 The Problem . . . 2
- 1.3 Environment . . . 2

2 Root Cause: `xsk_rcv_check()` **3**
- 2.1 The Redirect Call Chain . . . 3
- 2.2 Error Propagation and Silent Failure . . . 4
- 2.3 Documentation History . . . 4

3 Analysis: Why Cross-Queue Redirect Is Safe **5**
- 3.1 Driver Ring Index Management . . . 5
- 3.2 Buffer Lifecycle in `xsk_rcv_zc()` . . . 5
- 3.3 Teardown Safety . . . 6
- 3.4 DMA Coherency . . . 7
- 3.5 Error Handling . . . 7

4 Counterarguments Considered **7**
- 4.1 Fill Ring Depletion . . . 7
- 4.2 Unaligned Mode Pool Leaking . . . 7
- 4.3 Driver-Specific Complications . . . 8
- 4.4 The `bpf_redirect_map` API Confusion . . . 8

5 Proposed Kernel Patch **8**
5.1 Approach . . . 8
5.2 Proposed Diff . . . 8
5.3 What This Enables . . . 9
5.4 What This Does Not Change . . . 9

6 Testing Strategy **10**

7 Conclusion **10**

1 Motivation

1.1 The Multi-Queue AF_XDP Router

Modern AF_XDP applications use multiple hardware queues to exploit multi-core parallelism. Each queue has its own AF_XDP socket, NAPI context, and worker thread. With `XDP_SHARED_UMEM`, all sockets share a single memory-mapped region, eliminating cross-queue data copies.

A concrete use case is a multi-queue packet router performing Network Address Translation (NAT). Each queue owns a disjoint block of NAT port numbers and maintains lock-free state for reverse translation. When return traffic arrives from the internet, RSS distributes packets across all queues uniformly. Only the queue that *owns* the NAT port block for a given flow can perform efficient reverse NAT—ideally without cross-queue locking.

The natural solution is XDP-level steering: the XDP program inspects the destination port, looks up the owning queue in a BPF map, and calls `bpf_redirect_map(&xsks_map, owner_queue, 0)` to deliver the frame directly to the owning queue's AF_XDP socket.

1.2 The Problem

This does not work. The frame silently disappears:

- The BPF program's `bpf_redirect_map` returns `XDP_REDIRECT` (success from BPF's perspective).
- The XDP `packet_counters.redirected` counter increments.
- The frame never appears on the target socket's RX ring.
- No kernel log messages, no error counters, no tracepoint output visible without explicit `perf` instrumentation.

The failure is 100% reproducible and was confirmed by incremental bisection of the XDP program (Table 1).

Table 1: Bisection of XDP program modifications at the redirect label.

XDP code variant	**Result**
No steering (`queue_id = ctx->rx_queue_index`)	PASS
+ Parse L4 header, discard result	PASS
+ `bpf_map_lookup_elem(&nat_config_map, ...)`	PASS
+ `bpf_map_lookup_elem(&nat_block_map, &block_id)`	PASS
+ `queue_id = *owner` (cross-queue redirect)	**FAIL**

1.3 Environment

- Kernel: 6.17.0 (Ubuntu 25.10)
- NIC: Intel I210 Gigabit (`igb` driver)

- AF_XDP: 4 queues, `XDP_ZEROCOPY`, shared UMEM (single mmap, partitioned frame descriptors)
- XDP attachment: `BPF_LINK` (native/driver mode)

2 Root Cause: `xsk_rcv_check()`

The guard is in `net/xdp/xsk.c`:

Listing 1: The queue_id guard in xsk_rcv_check().

```
static int xsk_rcv_check(struct xdp_sock *xs,
                         struct xdp_buff *xdp, u32 len)
{
    if (!xsk_is_bound(xs))
        return -ENXIO;

    if (xs->dev != xdp->rxq->dev ||
        xs->queue_id != xdp->rxq->queue_index)
        return -EINVAL;  /* <-- blocks cross-queue */

    if (len > xsk_pool_get_rx_frame_size(xs->pool)
        && !xs->sg) {
        xs->rx_dropped++;
        return -ENOSPC;
    }

    return 0;
}
```

When a packet arrives on hardware queue X and the XDP program redirects to the socket bound to queue Y ($X \neq Y$), `xdp->rxq->queue_index` is X but `xs->queue_id` is Y. The check fails, returning `-EINVAL`.

2.1 The Redirect Call Chain

Figure 1 shows the complete call path from driver to the guard. The driver's involvement ends at `xdp_do_redirect()`—everything beyond that is generic kernel code in `net/core/filter.c` and `net/xdp/xsk.c`.

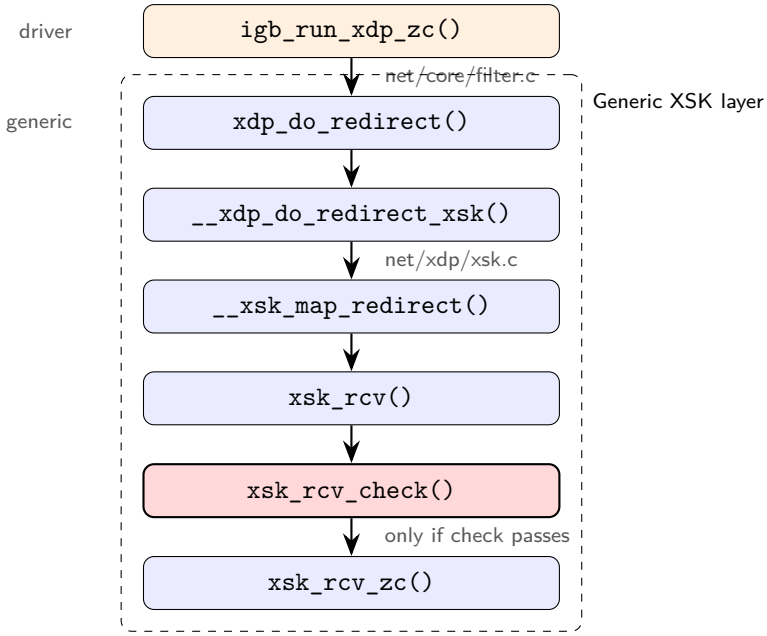

Figure 1: Call chain from driver NAPI handler to the queue_id guard. The driver (orange) only calls `xdp_do_redirect()`; all subsequent logic is in generic kernel code. The guard (red) is the sole point of failure for cross-queue redirect.

2.2 Error Propagation and Silent Failure

When `xsk_rcv_check()` returns `-EINVAL`, the error propagates back up the call chain. The only visibility is a tracepoint (`xdp:xdp_redirect_map_err`)—not a `pr_warn` or kernel log message. Unless the operator has `perf` attached to that tracepoint, the failure is completely invisible.

Back in the driver, `igb_run_xdp_zc()` checks the return value:

Listing 2: Error handling in igb_run_xdp_zc().

```
err = xdp_do_redirect(adapter->netdev, xdp, xdp_prog);
if (!err)
    return IGB_XDP_REDIR;

if (xsk_uses_need_wakeup(xsk_pool) && err == -ENOBUFS)
    result = IGB_XDP_EXIT;
else
    result = IGB_XDP_CONSUMED;
goto out_failure;
```

The redirect failure returns `IGB_XDP_CONSUMED`. The caller (`igb_clean_rx_irq_zc()`) then calls `xsk_buff_free(xdp)` to reclaim the buffer. The frame is properly freed—but from the BPF program's perspective, `bpf_redirect_map` already returned `XDP_REDIRECT`, so the BPF-side counter incremented. This creates a discrepancy between BPF counters (which show success) and actual delivery (which failed).

2.3 Documentation History

The AF_XDP maintainer Magnus Karlsson (Intel) briefly documented cross-queue redirect as working with shared UMEM in kernel 6.9 (commit `968595a93669`, February 2024), adding to `Documentation/networking/af_xdp.rst`:

> *"it is also possible to redirect to any socket as long as it is bound to the same umem with XDP_SHARED_UMEM"*

This was **reverted** in kernel 6.10 (commit `03e38d315f3c`, June 2024) after Yuval El-Hanany (Radware) reported that the documented behavior was inaccurate—`xsk_rcv_check()` still enforced the queue_id match. The documentation was aspirational, describing what the architecture supports rather than what the code permits.

The current kernel documentation explicitly states the limitation (`Documentation/bpf/map_xskmap.rst`):

> *"An AF_XDP socket that is bound to a certain ⟨netdev/queue_id⟩ will **only** accept XDP frames from that ⟨netdev/queue_id⟩."*

3 Analysis: Why Cross-Queue Redirect Is Safe

This section traces every stage of the redirect path and demonstrates that cross-queue delivery is correct for aligned shared-UMEM configurations without any driver modifications.

3.1 Driver Ring Index Management

The core of the safety argument rests on how the `igb` driver (representative of all ZC-capable drivers) manages buffer ownership through ring indices rather than per-slot state.

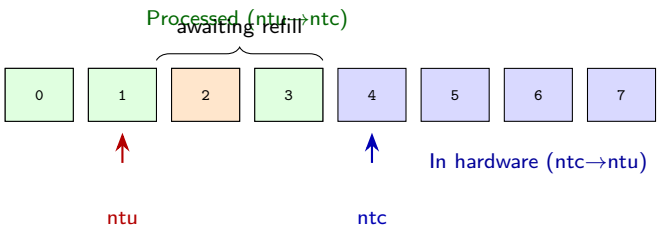

Figure 2: RX descriptor ring after processing a redirect at slot 2. The redirected slot (orange) is in the "processed" zone between `ntu` and `ntc`. It will be overwritten with a fresh buffer on the next refill cycle.

In `igb_clean_rx_irq_zc()`, when the XDP result is `IGB_XDP_REDIR`:

1. The driver does **not** call `xsk_buff_free(xdp)`—ownership has transferred to the generic redirect layer.
2. `ntc` (next-to-clean) advances past the slot.
3. The stale pointer in `rx_buffer_info_zc[slot]` is now in the "processed, awaiting refill" zone.
4. On refill, `igb_alloc_rx_buffers_zc()` calls `xsk_buff_alloc_batch()` which writes a **new** `xdp_buff` pointer into the slot, overwriting the stale one.

This is identical to the same-queue redirect path. The driver cannot distinguish between same-queue and cross-queue redirect—it calls `xdp_do_redirect()` and acts on the return value. The target queue is invisible to the driver.

3.2 Buffer Lifecycle in `xsk_rcv_zc()`

When a frame is successfully delivered (hypothetically, with the guard removed), the generic layer executes:

Listing 3: Zero-copy receive path in ___xsk_rcv_zc().

```
static int __xsk_rcv_zc(struct xdp_sock *xs,
        struct xdp_buff_xsk *xskb, u32 len, u32 flags)
{
```

```
    u64 addr;
    int err;

    addr = xp_get_handle(xskb, xskb->pool);
    err = xskq_prod_reserve_desc(xs->rx, addr, len, flags);
    if (err) {
        xs->rx_queue_full++;
        return err;
    }

    xp_release(xskb);
    return 0;
}
```

Three operations occur:

1. `xp_get_handle()` computes the UMEM offset from the `xdp_buff_xsk`. With shared UMEM, the offset is valid regardless of which queue's pool allocated the buffer—it's all the same mmap'd region.
2. `xskq_prod_reserve_desc()` writes the descriptor (UMEM address + length) to the target socket's RX ring. This is a simple ring buffer append with no queue affinity.
3. `xp_release()` for aligned pools is a **no-op**:

Listing 4: xp_release() is a no-op for aligned pools.

```
static inline void xp_release(struct xdp_buff_xsk *xskb)
{
    if (xskb->pool->unaligned)
        xskb->pool->free_heads[
            xskb->pool->free_heads_cnt++] = xskb;
    /* aligned mode: nothing to do */
}
```

For aligned pools (the common case, including any configuration using the default 4096-byte chunk size), `xdp_buff_xsk` structures are statically indexed by `addr / chunk_size` in the pool's `heads[]` array. They are not dynamically allocated or freed—they are looked up fresh from the fill queue address each time `xp_get_xskb()` is called during buffer allocation.

3.3 Teardown Safety

A concern was raised that ring teardown could double-free redirected buffers. The teardown function `igb_clean_rx_ring_zc()` iterates only from `ntc` to `ntu`—the range of buffers currently posted to hardware and not yet processed:

Listing 5: Ring teardown only frees unprocessed buffers.

```
void igb_clean_rx_ring_zc(struct igb_ring *rx_ring)
{
    u16 ntc = rx_ring->next_to_clean;
    u16 ntu = rx_ring->next_to_use;

    while (ntc != ntu) {
        struct xdp_buff *xdp =
            rx_ring->rx_buffer_info_zc[ntc];
        xsk_buff_free(xdp);
        ntc++;
        if (ntc >= rx_ring->count)
            ntc = 0;
    }
}
```

A redirected buffer has already been processed (`ntc` advanced past its slot in `igb_clean_rx_irq_zc()`). It is therefore outside the `ntc`→`ntu` range and will not be freed. Furthermore, teardown cannot race with NAPI because `igb_txrx_ring_disable()` calls `napi_disable()` before touching ring state, ensuring `ntc` reflects all completed processing.

3.4 DMA Coherency

With shared UMEM, DMA mappings are established at the pool level via `xsk_pool_dma_map()`, not per-buffer. Each queue's pool maps the same UMEM pages to the same device, producing identical DMA addresses for the same UMEM offsets. The per-buffer CPU cache sync (`xsk_buff_dma_sync_for_cpu()`) is performed on the source queue before the XDP program executes—before any redirect decision. No additional sync is needed for cross-queue delivery, as userspace on the target queue simply reads from CPU-coherent memory.

3.5 Error Handling

The driver correctly handles redirect failures. As shown in Listing 2, `igb_run_xdp_zc()` checks `xdp_do_redirect()`'s return value. On failure (e.g., target RX ring full), the buffer is freed via `xsk_buff_free()` in the `IGB_XDP_CONSUMED` path. No buffer leaks occur on redirect failure, whether same-queue or cross-queue.

4 Counterarguments Considered

Several potential objections to relaxing the guard were raised during analysis. We address each in turn.

4.1 Fill Ring Depletion

Concern: If frames consistently migrate from queue X to queue Y via cross-queue redirect, queue X's fill ring drains over time (frames allocated from X's fill ring are consumed by Y's RX ring and returned to Y's fill ring by userspace).

Response: This is an application-level resource management concern, not a kernel invariant violation. With shared UMEM, all UMEM addresses are valid on any queue's fill ring. A well-designed application maintains a global free pool of frame descriptors and redistributes them across fill rings based on demand. Applications that use `XDP_SHARED_UMEM` are already managing multi-queue buffer lifecycle; cross-queue frame migration is a natural extension.

The kernel does not prevent userspace from submitting "foreign" UMEM addresses to any fill ring today. The fill ring consumer (`xp_alloc_new_from_fq()`) accepts any valid UMEM offset.

4.2 Unaligned Mode Pool Leaking

Concern: For unaligned pools (`XDP_UMEM_UNALIGNED_CHUNK_FLAG`), `xp_release()` adds the `xdp_buff_xsk` back to `pool->free_heads[]`. Cross-queue redirect would return xskb entries to the source pool's free list, while the target pool's `free_heads_cnt` would shrink over time.

Response: This is a valid concern for unaligned mode and would need to be addressed separately. However, aligned mode (the default and overwhelmingly common configuration) is unaffected because `xp_release()` is a no-op—`xdp_buff_xsk` structures are statically indexed and never moved between free lists.

A proposed patch could initially restrict cross-queue redirect to aligned pools and address unaligned mode in a follow-up.

4.3 Driver-Specific Complications

Concern: Drivers maintain hidden per-queue state beyond the ring indices, and cross-queue redirect could corrupt this state.

Response: In ZC mode, the driver's per-buffer state is exactly one pointer: `rx_buffer_info_zc[slot]`. There is no per-buffer DMA mapping, no page reference, no metadata structure. The pointer is overwritten on refill with a fresh allocation. The driver has no "hidden accounting" beyond the ring indices.

This was verified for `igb` and holds for all upstream ZC drivers (`ice`, `igc`, `i40e`, `mlx5`), which follow the same pattern: read `xdp_buff` from slot, run XDP, advance index on redirect, refill with new allocation.

Critically, the driver never learns the target queue of a redirect. It calls `xdp_do_redirect()` and gets back success or failure. The generic XSK layer handles all target-queue logic. Cross-queue redirect requires zero changes to any driver.

4.4 The `bpf_redirect_map` API Confusion

Concern: Why does `bpf_redirect_map(&xsks_map, key, 0)` accept a key that can differ from `ctx->rx_queue_index` if cross-queue delivery doesn't work?

Response: `bpf_redirect_map` is a generic helper shared across three map types (Table 2). For `DEVMAP` and `CPUMAP`, cross-target redirect is the entire purpose. For `XSKMAP`, the key is a map lookup index that happens to conventionally equal the queue index. The BPF verifier cannot distinguish the map type at verification time, so it accepts any key. The API ambiguity is an artifact of generality, not a feature.

Table 2: Behavior of `bpf_redirect_map` across map types.

Map type	Index meaning	Cross-target?
`BPF_MAP_TYPE_DEVMAP`	target netdev	Yes
`BPF_MAP_TYPE_CPUMAP`	target CPU	Yes
`BPF_MAP_TYPE_XSKMAP`	socket map key	**No** (guard)

5 Proposed Kernel Patch

5.1 Approach

Relax the `queue_id` check in `xsk_rcv_check()` to allow cross-queue redirect when the source and target sockets share the same UMEM and the pool uses aligned chunks. The check remains in place for:

- Cross-device redirect (different `netdev`)—remains blocked.
- Non-shared-UMEM sockets—remains blocked.
- Unaligned mode—remains blocked (pending separate analysis).

5.2 Proposed Diff

The removed lines are shown in red and added lines in green:

Listing 6: Proposed change to xsk_rcv_check().

```
static int xsk_rcv_check(struct xdp_sock *xs,
                         struct xdp_buff *xdp, u32 len)
{
```

```
     if (!xsk_is_bound(xs))
         return -ENXIO;
-    if (xs->dev != xdp->rxq->dev ||
-        xs->queue_id != xdp->rxq->queue_index)
+    if (xs->dev != xdp->rxq->dev)
+        return -EINVAL;
+
+    if (xs->queue_id != xdp->rxq->queue_index &&
+        !xsk_pool_same_umem(xs->pool, xdp))
         return -EINVAL;

     if (len > xsk_pool_get_rx_frame_size(xs->pool)
         && !xs->sg) {
         xs->rx_dropped++;
         return -ENOSPC;
     }

     return 0;
 }
```

Where `xsk_pool_same_umem()` is a new helper:

Listing 7: Helper to check shared UMEM between source and target.

```
static inline bool xsk_pool_same_umem(
    struct xsk_buff_pool *target_pool,
    struct xdp_buff *xdp)
{
    struct xsk_buff_pool *src_pool;

    src_pool = xdp->rxq->dev->_rx[
        xdp->rxq->queue_index].pool;
    if (!src_pool || !target_pool)
        return false;

    return target_pool->umem == src_pool->umem
        && !target_pool->unaligned;
}
```

5.3 What This Enables

With this change, an XDP program can redirect a packet arriving on any hardware queue to any AF_XDP socket on the same device, provided all sockets share the same UMEM via `XDP_SHARED_UMEM` with aligned chunks. The application receives the frame on the target socket's RX ring and is responsible for returning the UMEM frame descriptor to an appropriate fill ring.

5.4 What This Does Not Change

- No driver code is modified.
- No new BPF helpers or map types are introduced.
- The existing `bpf_redirect_map` API is used as-is.
- Cross-device redirect remains blocked.
- Unaligned UMEM remains blocked (future work).
- Applications that do not use cross-queue redirect are unaffected.

6 Testing Strategy

A patch submission should include:

1. **Selftest extension:** Extend `tools/testing/selftests/bpf/prog_tests/xdp_do_redirect.c` with a cross-queue shared-UMEM test case.
2. **xsk selftest:** Extend `tools/testing/selftests/bpf/xskxceiver.c` with cross-queue redirect scenarios (same device, shared UMEM, aligned chunks).
3. **Negative tests:** Verify that cross-queue redirect still fails for non-shared UMEM and unaligned mode.
4. **Stress testing:** Sustained cross-queue traffic to verify no fill ring depletion, buffer leaks, or pool corruption under load. The application must actively redistribute frame descriptors.
5. **Multi-driver validation:** Test on at least `igb`, `igc`, and `ice` to confirm no driver-specific regressions.

7 Conclusion

The `queue_id` guard in `xsk_rcv_check()` is the sole barrier to cross-queue XDP redirect with AF_XDP shared UMEM in zero-copy mode. Tracing the complete code path from driver NAPI handler through generic XSK infrastructure demonstrates that:

- Driver ring management correctly handles buffer ownership transfer for redirected frames (same mechanism as same-queue redirect).
- No driver modifications are required—the driver is unaware of the redirect target.
- DMA coherency is maintained at the UMEM/pool level.
- Teardown and error paths are safe.
- For aligned pools, `xdp_buff_xsk` accounting is statically indexed and cannot leak across pools.

The guard was appropriate as a conservative default when AF_XDP was introduced, and it remains correct for non-shared-UMEM and unaligned configurations. However, for the aligned shared-UMEM case—which represents the majority of multi-queue AF_XDP deployments—the restriction prevents a natural and architecturally sound use of XDP redirect for intra-application packet steering.

We propose a minimal, targeted relaxation: allow cross-queue redirect when the source and target sockets demonstrably share the same UMEM with aligned chunks. This change is confined to a single function in generic XSK code, requires no driver modifications, and is gated behind conditions that ensure buffer accounting integrity.